R1.01

CM-04

Algorithmique - récursivité

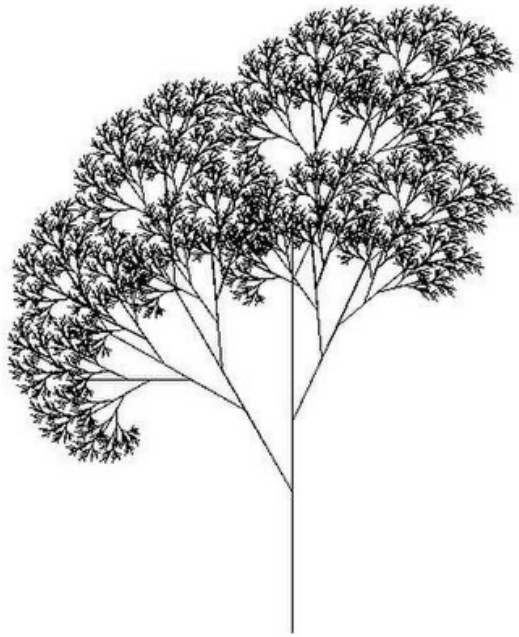
Basé sur le cours de Loïg Jezequel

Définition

Wikipedia :

Un algorithme récursif est un algorithme qui résout un problème en calculant des solutions d'instances plus petites du même problème.

Un arbre est composé de sous-arbres qui sont des arbres.



Exemple : recherche dichotomique

```go
package iterative

func rechercheDicho(x int, t []int) int {
    debut := 0
    fin := len(t)
    var milieu int
    for debut < fin {
      milieu = (debut + fin) / 2
      if t[milieu] == x {
        return milieu
      } else {
        if t[milieu] > x {
          fin = milieu
        } else {
          debut = milieu + 1
        }
      }
    }
    return -1
}
```

```go
package recursive

func rechercheDicho(x int, t []int) int {
    if len(t) == 0 {
      return -1
    }
    milieu := len(t) / 2
    if t[milieu] == x {
      return milieu
    }
    if t[milieu] > x {
      return rechercheDicho(x, t[:milieu])
    }
    tmp := rechercheDicho(x, t[milieu+1:])
    if tmp == -1 {
      return -1
    }
    return milieu + tmp
}
```

Principe

1. Il faut au moins un cas d’arrêt
2. Il faut à chaque appel récursif converger vers un point d’arrêt

Rem : On suppose lors de l’appel récursif que l’étape précédente fonctionne

Principe

1. Il faut au moins un cas d’arrêt

```
if len(t) == 0 {
    return -1
  }
```

On peut statuer, il n’y a pas d’appel récursif.

Principe

2. Il faut à chaque appel récursif converger vers un point d'arrêt

Ici le point d'arrêt est défini par une taille de 0

recherchеDicho(x, t[:milieu]) et recherchеDicho(x, t[milieu+1:])

réduisent la taille de t et donc convergent vers 0

Rem: a[low : high], élément de position low inclus et de position high exclus

Factorielle

- Définition

  Fac(n) =

  si n >1 (n-1) (n-2) … 1

  sinon 1

- Condition d'arrêt ?
- Appel récursif ?

Fac(0) = 1

Fac(1) = 1

Fac(2) = 2 x 1 = 2

Fac(3) = 3 x 2 x 1 = 6

Fac(4) = 4 x 3 x 2 x 1 = 24

Factorielle

- Définition

  Fac(n) =

  si n >1 (n-1) (n-2) … 1

  sinon 1

- Condition d'arrêt

  f(0)=f(1)=1

```
if n < 2 {
    return 1
}
```

- Appel récursif ?

Factorielle

- Définition

  Fac(n) =

  si n >1 (n-1) (n-2) … 1

  sinon 1

- Condition d'arrêt

  f(0)=f(1)=1

```
if n < 2 {
    return 1
}
```

- Appel récursif

Fac(n) = n (n-1) (n-2) 2

Fact(n-1) = (n-1) (n-2) 2

Fac(n) = n fac(n-1)

Factorielle

```
func factorielle(n int) int {
    if n < 2 {
      return 1
    }
    return n *
  factorielle(n-1)
  }
```

factorielle(3) =

3 * factorielle(2)

factorielle(2) = 2 * factorielle(1)

factorielle(1) =1

Suites récursives

Une suite est une famille d'éléments (typiquement des réels) appelés termes, indexé par les entiers naturels. On la note en général (U_n) et on note son n-ième terme u_n.

Définition par le terme général

On définit u_n en fonction de n.

Exemples : $u_n = n$, $u_n = n!$, $u_n = 1/n$, $u_n = 2 * n$, etc.

Définition récursive : On définit u_n en fonction des termes précédents $(u_0, u_1, \ldots, u_{n-1})$.

Dans ce cas on doit initialiser la suite : donner la valeur de ses premiers termes

Exemples :

$u_n = n \times u_{n-1}$ avec $u_0 = 1$,

$u_n = u_{n-1} + u_{n-2}$ avec $u_0 = 0$ et $u_1 = 1$

Suites récursives : simples - calcul itératif

Suite simple :

On considère une suite (u_n) telle que le terme un ne dépend que du terme u_{n-1} : $u_n = f\,(_{un-1})$.

Algorithme (calcul de u_n)

Soit i = 0 et soit u_i = u0.

Tant que i < n, poser

$u_i = f\,(u_i\,)$

puis ajouter 1 à i.

Retourner u_i.

Exemple : on veut calculer u_5 lorsque $u_n = 2 \times u_{n-1} + 1$ et $u_0 = 0$.

Suites récursives : simples – calcul itératif

Algorithme (calcul de u_n)

Soit i = 0 et soit u_i = u0.

Tant que i < n, poser

$u_i = f(u_i)$

puis ajouter 1 à i.

Retourner u_i.

Exemple : on veut calculer u_5 lorsque $u_n = 2 \times u_{n-1} + 1$ et $u_0 = 0$.

$u_i = 2 \times 0 + 1 = 1$, i = 1

$u_i = 2 \times 1 + 1 = 3$, i = 2

$u_i = 2 \times 3 + 1 = 7$, i = 3

$u_i = 2 \times 7 + 1 = 15$, i = 4

$u_i = 2 \times 15 + 1 = 31$, i = 5

i ≥ 5, on retourne $u_i = 31$

Suites récursives : simples - calcul récursif

Suite simple :

On considère une suite (u_n) telle que le terme un ne dépend que du terme u_{n-1} : $u_n = f\,(_{un-1})$.

Algorithme (calcul de u_n)

On appelle u(i) la fonction

qui retourne u_0 si i = 0 et

qui retourne f (u(i– 1)) sinon

Retourner u(n).

Exemple : on veut calculer u_5 lorsque $u_n = 2 \times u_{n-1} + 1$ et $u_0 = 0$.

Suites récursives : simples - calcul récursif

Algorithme (calcul de u_n)

On appelle u(i) la fonction

qui retourne u_0 si i = 0 et

qui retourne f (u(i− 1)) sinon

Retourner u(n).

u(5) = 2*u(4) + 1

u(4) = 2* u(3) + 1

…

u(0)=0

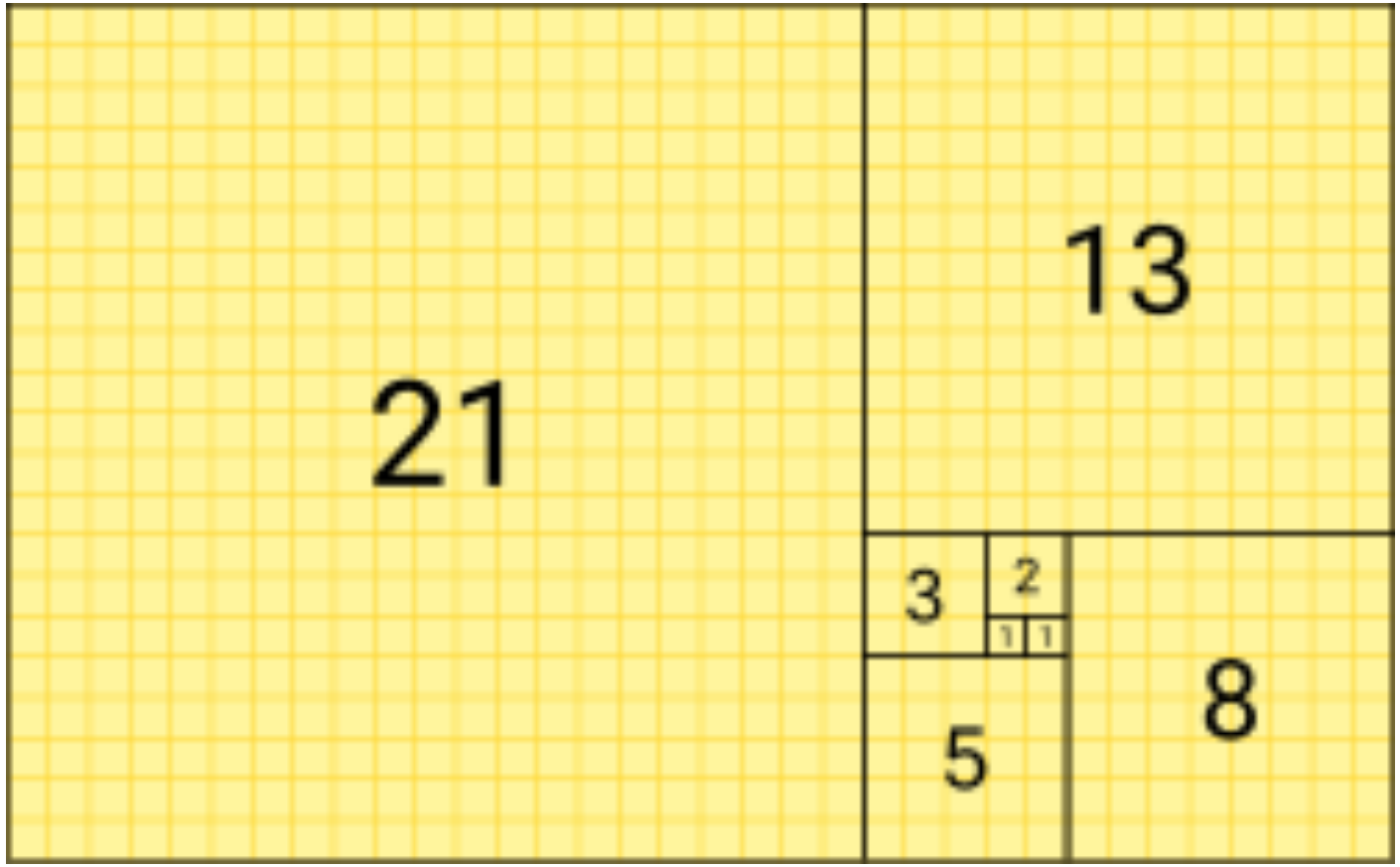
Exemple : on veut calculer u_5
lorsque $u_n = 2 \times u_{n-1} + 1$ et $u_0 = 0$.

Autre suite : Fibonacci

F(0)=0

F(1)=1

F(n) = F(n-1) + F(n-2) pour n >1



Autre suite : Fibonacci

```
package recursive

  func fibonnaci(n uint) uint {
     if n == 0 {
        return 0
     }
     if n == 1 {
        return 1
     }

     return fibonnaci(n-1) + fibonnaci(n-2)
  }
```

Autre problème : palindrome

Un palindrome est une chaîne de caractères qui se lit exactement pareil dans les deux sens.

Exemple : kayak, tôt, été, radar, ressasser

- Condition d’arrêt ?
- Apple récursif ?

Autre problème : palindrome

Un palindrome est une chaîne de caractères qui se lit exactement pareil dans les deux sens.

Exemple : kayak, tôt, été, radar, ressasser

- Condition d'arrêt ?
  - Chaine vide
  - Premier et dernier caractères différents
- Apple récursif ?
  - Mot privé du premier et du dernier caractère

Autre problème : palindrome

Un palindrome est une chaîne de caractères qui se lit exactement pareil dans les deux sens.

```
package recursive

  func estPalindrome(s string) (b bool) {
    if len(s) <= 1 {
      return true
    }
    if s[0] != s[len(s)-1] {
      return false
    }
    return estPalindrome(s[1 : len(s)-1])
  }
```

Problème plus complexes

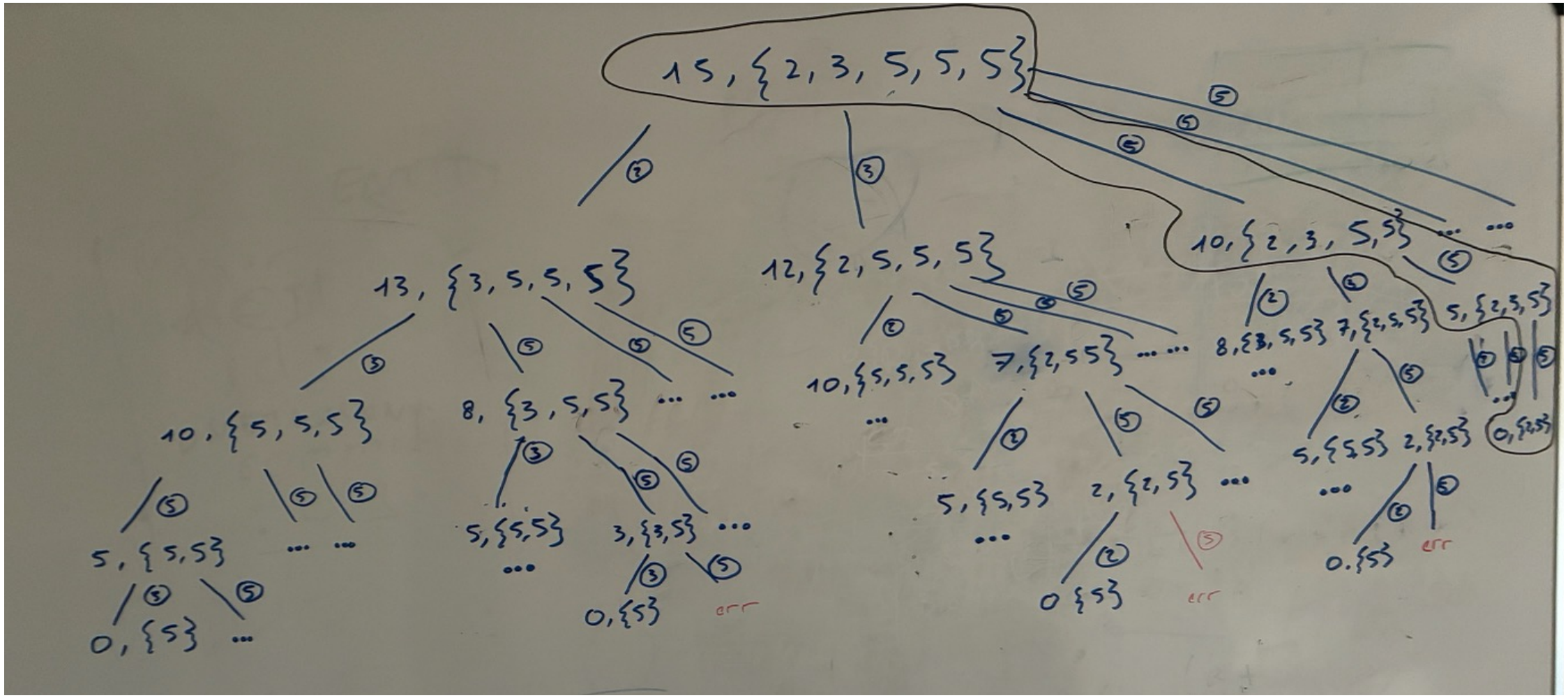
Rendre la monnaie avec le moins de pièces possibles.

Rendre 15 avec {2, 3, 5, 5, 5} => {5,5,5}

Rendre 15 avec {2,3,5}=> impossible

Idée ?

Problème plus complexes



A propos de la récursivité

Gros avantage : Les solutions à certains problèmes s'écrivent très simplement de manière récursive mais sont compliquées à écrire correctement de manière itérative (par des boucles).

Exemple (code Go) : étant donné un tableau t contenant des caractères et un entier

k ≤ len(t), donner toutes les chaînes de k caractères que l'on peut construire en utilisant au plus une fois chaque caractère de t.

Pour t = |a | b |c |d | et k = 2 on doit trouver ab, ac, ad, ba, bc, bd, ca, cb, cd, da, db et dc.

Pour t = |a | b |c |d | et k = 3 on doit trouver abc, abd, acb, acd, adb, adc, bac, bad, bca, bcd, bda, bdc, cab, cad, cba, cbd, cda, cdb, dab, dac, dba, dbc, dca, dcb.

Autres exemples plus tard : Tri fusion, algorithmes sur les arbres

A propos de la récursivité

- Ressources :

Un appel de fonction est plus long qu'un tour de boucle.

Les appels de fonctions non résolus occupent la mémoire.

- Récursivité terminale

Si l'appel récursif est la dernière chose qui se passe dans une fonction, le compilateur peut optimiser et ´éviter les appels de fonctions non résolus.

Terminale : **renvoie** récursionTerminale(n - 1)

Non Terminale : **renvoie** n + récursionNonTerminale(n - 1) //On doit conserver la référence au résultats précédents

Remarque

- Une fonction récursive peut toujours être rendue non récursive
- terminale en utilisant la notion de continuation. Cependant, c'est
- un peu compliqué et on n'en parlera pas dans ce cours.